

Lab Assignment 5

CMPE 146 Section 1

Professor Bhawandeep Singh Harsh

Andrew Bertch

Robert Trinh

Table of Contents

Exercise 1 – RC oscillator	3
Exercise 2 – Frequency measurement.....	4
Exercise 3 – LED control	8
Exercise 4 – Other oscillators	10
Appendix A – MCU Source Code.....	13
oscillate.c.....	13
measure_frequency.c.....	15
touch_LED.c	18
multiple_oscillators.c.....	23

Exercise 1 – RC oscillator

In this exercise, we create a simple RC oscillator using one of the GPIO pins on the TI Launchpad. We can use the GPIO pin's pull-up pull-down resistor for the resistance and the parasitic capacitances in the MCUs and boards wiring for the capacitance. By changing between either pull-down or pull-up on the pin, we can charge and discharge the capacitance creating an oscillator. This switch will occur when the pin reads high or low. Below is the console output from implementing the oscillator scheme above.

CORTEX_MoP:

```
0101010101010101010101010101010101010101010101010101010101010101
01010101010101010101010101
```

An alternative method could be achieved using the timer chip. We could decide a frequency that we want our oscillator to switch at. We could then poll the timer chip until the time elapses. A variable can then be switched from 0 or to 1 to keep track of the current state of the oscillator.

Exercise 2 – Frequency measurement

In this exercise, we measure the frequency of the oscillator implemented in Exercise 1. We will also measure how touching the pin with a finger changes the frequency of the oscillator. Each cycle gets measured and printed to the console with a 1 second delay between measured cycles. The measured frequencies are shown below with tests 1-10 being touching the side of the pin with pressure, tests 11-20 touching the side of the pin with no pressure, tests 21-30 touching the top of the pin with pressure, tests 31-40 touching the top of the pin with no pressure, and tests 41-50 not touching the pin.

CORTEX_MoP: Test 1 starting now
58MHz

CORTEX_MoP: Test 9 starting now
66MHz

CORTEX_MoP: Test 2 starting now
59MHz

CORTEX_MoP: Test 10 starting now
66MHz

CORTEX_MoP: Test 3 starting now
60MHz

CORTEX_MoP: Test 11 starting now
37MHz

CORTEX_MoP: Test 4 starting now
59MHz

CORTEX_MoP: Test 12 starting now
45MHz

CORTEX_MoP: Test 5 starting now
63MHz

CORTEX_MoP: Test 13 starting now
49MHz

CORTEX_MoP: Test 6 starting now
66MHz

CORTEX_MoP: Test 14 starting now
48MHz

CORTEX_MoP: Test 7 starting now
58MHz

CORTEX_MoP: Test 15 starting now
49MHz

CORTEX_MoP: Test 8 starting now
64MHz

CORTEX_MoP: Test 16 starting now
53MHz

CORTEX_MoP: Test 17 starting now
49MHz

CORTEX_MoP: Test 27 starting now
46MHz

CORTEX_MoP: Test 18 starting now
52MHz

CORTEX_MoP: Test 28 starting now
47MHz

CORTEX_MoP: Test 19 starting now
53MHz

CORTEX_MoP: Test 29 starting now
43MHz

CORTEX_MoP: Test 20 starting now
54MHz

CORTEX_MoP: Test 30 starting now
34MHz

CORTEX_MoP: Test 21 starting now
37MHz

CORTEX_MoP: Test 31 starting now
27MHz

CORTEX_MoP: Test 22 starting now
45MHz

CORTEX_MoP: Test 32 starting now
27MHz

CORTEX_MoP: Test 23 starting now
42MHz

CORTEX_MoP: Test 33 starting now
27MHz

CORTEX_MoP: Test 24 starting now
48MHz

CORTEX_MoP: Test 34 starting now
28MHz

CORTEX_MoP: Test 25 starting now
48MHz

CORTEX_MoP: Test 35 starting now
28MHz

CORTEX_MoP: Test 26 starting now
44MHz

CORTEX_MoP: Test 36 starting now
28MHz

CORTEX_MoP: Test 37 starting now
29MHz

CORTEX_MoP: Test 38 starting now
29MHz

CORTEX_MoP: Test 39 starting now
28MHz

CORTEX_MoP: Test 40 starting now
29MHz

CORTEX_MoP: Test 41 starting now
13MHz

CORTEX_MoP: Test 42 starting now
13MHz

CORTEX_MoP: Test 43 starting now
13MHz

CORTEX_MoP: Test 44 starting now
13MHz

CORTEX_MoP: Test 45 starting now
13MHz

CORTEX_MoP: Test 46 starting now
13MHz

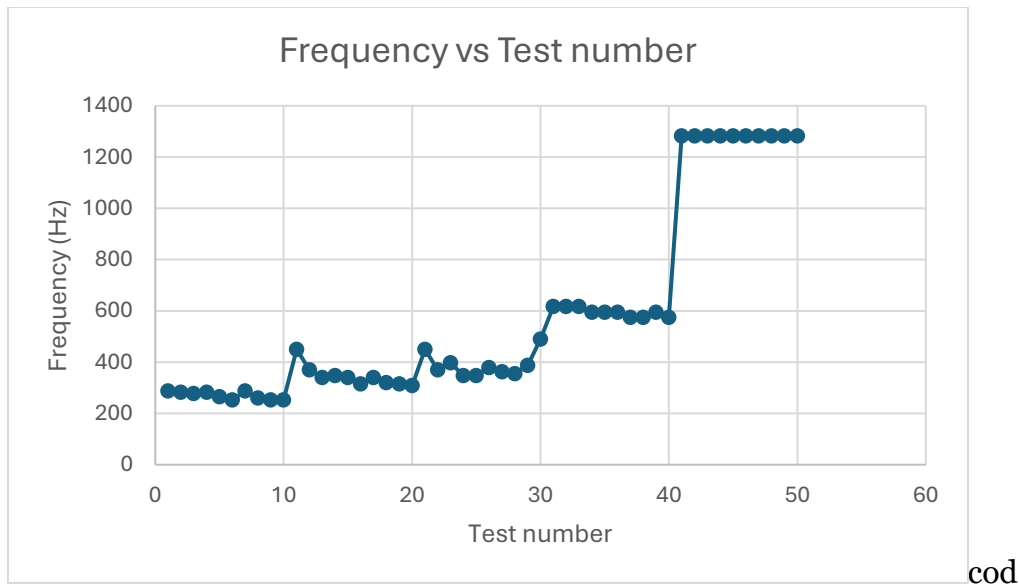
CORTEX_MoP: Test 47 starting now
13MHz

CORTEX_MoP: Test 48 starting now
13MHz

CORTEX_MoP: Test 49 starting now
13MHz

CORTEX_MoP: Test 50 starting now
13MHz

Something to note, the data collected was end time minus start time divided by 60. This is not the frequency and in my tables and graph was corrected by multiplying the data by 60 and then doing 1 divided by end minus start. Since $f=1/T$ and end minus start is the period. The time was also measured in microseconds so in excel the formula was $1/(T*1e-6)$.



Graph 2.1

Graph 2.1 plots the frequency measurements from the console. By plotting the this data it can be seen that touching the pin with more of our finger and with greater finger pressure the frequency of the oscillator goes down. That is, the oscillator is slower with more of a finger touching the pin. This is due to the capacitance of the circuit going up increasing the time constant of the discharge rate of the circuit.

The maximum speed of can be estimated by measuring a single charge to discharge cycle and cutting out all other extra lines of code. The implementation currently gets close but could see the frequency go as high as ~1400 Hz.

Exercise 3 – LED control

In this exercise, we use capacitive touch control to turn on and off an LED. We make use of some of the code from the previous two exercises to measure and detect if a finger is touching to toggle a LED.

```
for(int i = 0; i < 5; i++)
{
    printf("Test %d starting now ", test + 1);
    start = DL_Timer_getTimerCount(TIMG12);
    while (count < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 0)
        {
            count++;
            prev = 0;
            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
        }
        if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 1) {
            count++;
            prev = 1;
            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
        }
    }
    end = DL_Timer_getTimerCount(TIMG12);
    count = 0;
```

```
    prev = -1;

    test++;

    frequencySum += (end - start) / 60;
    printf("%dhz\n", (end - start) / 60);

    delay_cycles(32000000);
}
```

This code snippet does a measurement of the initial parameters to set a threshold value. As previously noted, the console outputs and variables are noted as being a frequency but are in fact the period. The console outputs that this generates as well as the calculated threshold value is shown below.

CORTEX_MoP: Test 1 starting now 15hz

CORTEX_MoP: Test 2 starting now 15hz

CORTEX_MoP: Test 3 starting now 15hz

CORTEX_MoP: Test 4 starting now 15hz

CORTEX_MoP: Test 5 starting now 15hz

CORTEX_MoP: Calibration threshold: 15hz

A video of this exercise functioning is included [here](#).

Exercise 4 – Other oscillators

In this exercise, we looked at oscillators on other GPIO pins. We run the same code from exercise 2 on 3 different pins and compared the measured period in microseconds divided by 60. This can be converted to frequency by multiplying by 60 and dividing 1 by the value. The console outputs are noted as being the frequency but are once again the period of the oscillator. The console outputs of this exercise are included below.

CORTEX_MoP: Test 1 starting now 14hz

CORTEX_MoP: Test 2 starting now 14hz

CORTEX_MoP: Test 3 starting now 14hz

CORTEX_MoP: Test 4 starting now 14hz

CORTEX_MoP: Test 5 starting now 14hz

CORTEX_MoP: Average Frequency PB9: 14hz

CORTEX_MoP: : Test 1 starting now 16hz

CORTEX_MoP: Test 2 starting now 16hz

CORTEX_MoP: Test 3 starting now 16hz

CORTEX_MoP: Test 4 starting now 16hz

CORTEX_MoP: Test 5 starting now 16hz

CORTEX_MoP: Average Frequency PB5: 16hz

CORTEX_MoP: : Test 1 starting now 17hz

CORTEX_MoP: Test 2 starting now 17hz

CORTEX_MoP: Test 3 starting now 17hz

CORTEX_MoP: Test 4 starting now 17hz

CORTEX_MoP: Test 5 starting now 17hz

CORTEX_MoP: Average Frequency PB11: 17hz

CORTEX_MoP: : Test 1 starting now 18hz

CORTEX_MoP: Test 2 starting now 18hz

CORTEX_MoP: Test 3 starting now 18hz

CORTEX_MoP: Test 4 starting now 18hz

CORTEX_MoP: Test 5 starting now 18hz

CORTEX_MoP: Average Frequency PB21: 18hz

The frequencies are likely different due to differences in the internal circuit capacitances for each pin. Each pin is connected to other chips and circuits on the board. This other devices and their supporting capacitors and traces can increase the internal capacitance

and increase the charge discharge time of the RC circuit present on each pin assuming equal resistor values.

Appendix A – MCU Source Code

oscillate.c

```
#include "ti_msp_dl_config.h"

#include <stdio.h>

int count = 0;\
int prev = -1;

int main(void)
{
    SYSCFG_DL_init();

    DL_GPIO_enablePower(GPIOB);
    DL_GPIO_initDigitalInput(IOMUX_PINCM26);
    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM26,
    IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
    DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
    DL_GPIO_WAKEUP_DISABLE);

    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_9);

    while (1) {
        while (count < 100) {
            if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 0)
            {
                count++;
                prev = 0;
            }
        }
    }
}
```

```

        printf("0");

        DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

    }

    if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 1) {

        count++;

        prev = 1;

        printf("1");

        DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

    }

}

count = 0;

printf("\n");

}

}

```

measure_frequency.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int count = 0;
```

```
int test = 0;
```

```
int prev = -1;
```

```
int start, end;
```

```
int main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    DL_GPIO_enablePower(GPIOB);
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM26);
```

```
    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM26,  
IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_9);
```

```
    DL_Timer_enablePower(TIMG12);
```

```
    DL_Timer_ClockConfig config;
```

```
    config.clockSel = DL_TIMER_CLOCK_BUSCLK;
```

```
    config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
```

```
    config.prescale = 0;
```

```

DL_Timer_setClockConfig(TIMG12, &config);
DL_Timer_TimerConfig timerConfig;
timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
timerConfig.period = -1;
timerConfig.startTimer = DL_TIMER_START;
timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;
timerConfig.counterVal = 0;
DL_Timer_initTimerMode(TIMG12, &timerConfig);

while (test < 50) {
    printf("Test %d starting now ", test + 1);
    start = DL_Timer_getTimerCount(TIMG12);
    while (count < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 0)
        {
            count++;
            prev = 0;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
        }

        if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 1) {
            count++;
            prev = 1;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

```

```
        }  
    }  
    end = DL_Timer_getTimerCount(TIMG12);  
    count = 0;  
    prev = -1;  
    test++;  
  
    printf("%dhz\n", (end - start) / 60);  
  
    delay_cycles(32000000);  
}  
  
while (1) {  
  
    }  
}
```

touch_LED.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int count = 0;
```

```
int test = 0;
```

```
int prev = -1;
```

```
int start, end;
```

```
int frequencySum;
```

```
int thresholdParameter;
```

```
int main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    DL_GPIO_enablePower(GPIOB);
```

```
    DL_GPIO_initDigitalOutput(IOMUX_PINCM57); // PB26, red LED in LED2
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM26);
```

```
    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM26,  
IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM57); // PB26, red LED
```

```
    DL_GPIO_enableOutput(GPIOB, DL_GPIO_PIN_26); // PB26, red LED
```

```

DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_9);
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);


DL_Timer_enablePower(TIMG12);
DL_Timer_ClockConfig config;
config.clockSel = DL_TIMER_CLOCK_BUSCLK;
config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
config.prescale = 0;
DL_Timer_setClockConfig(TIMG12, &config);
DL_Timer_TimerConfig timerConfig;
timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
timerConfig.period = -1;
timerConfig.startTimer = DL_TIMER_START;
timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;
timerConfig.counterVal = 0;
DL_Timer_initTimerMode(TIMG12, &timerConfig);


for(int i = 0; i < 5; i++)
{
    printf("Test %d starting now ", test + 1);
    start = DL_Timer_getTimerCount(TIMG12);
    while (count < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 0)
        {
            count++;

```

```

        prev = 0;

        DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
    }

    if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 1) {
        count++;

        prev = 1;

        DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
    }
}

end = DL_Timer_getTimerCount(TIMG12);
count = 0;
prev = -1;
test++;

frequencySum += (end - start) / 60;
printf("%dhz\n", (end - start) / 60);

delay_cycles(32000000);
}

thresholdParameter = frequencySum / 5;

printf("Calibration threshold: %dhz\n", thresholdParameter);

```

```

int startControl, endControl;

int frequency;

bool isLEDOOn = false;

while(1)
{
    startControl = DL_Timer_getTimerCount(TIMG12);
    while (count < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 0)
        {
            count++;

            prev = 0;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
        }

        if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prev != 1) {
            count++;

            prev = 1;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
        }
    }

    endControl = DL_Timer_getTimerCount(TIMG12);

    count = 0;
}

```

```

prev = -1;

frequency = (endControl - startControl) / 60;

if((frequency > thresholdParameter))
{
    if(!isLEDOn)
    {
        DL_GPIO_setPins(GPIOB, DL_GPIO_PIN_26);
        isLEDOn = true;
    }
    else if(isLEDOn)
    {
        DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_26);
        isLEDOn = false;
    }
}

delay_cycles(8000000);

}

}

```

multiple_oscillators.c

```
#include "ti_msp_dl_config.h"
```

```
#include <stdio.h>
```

```
int countPB9 = 0;
```

```
int testPB9 = 0;
```

```
int prevPB9 = -1;
```

```
int countPB5 = 0;
```

```
int testPB5 = 0;
```

```
int prevPB5 = -1;
```

```
int countPB11 = 0;
```

```
int testPB11 = 0;
```

```
int prevPB11 = -1;
```

```
int countPB21 = 0;
```

```
int testPB21 = 0;
```

```
int prevPB21 = -1;
```

```
int startPB9, endPB9;
```

```
int startPB5, endPB5;
```

```
int startPB11, endPB11;
```

```
int startPB21, endPB21;
```

```
int frequencySumPB9, frequencySumPB5, frequencySumPB11,  
frequencySumPB21;
```

```
int averageFrequencyPB9, averageFrequencyPB5, averageFrequencyPB11,  
averageFrequencyPB21;
```

```
int main(void)
```

```
{
```

```
    SYSCFG_DL_init();
```

```
    DL_GPIO_enablePower(GPIOB);
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM26);
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM18);
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM28);
```

```
    DL_GPIO_initDigitalInput(IOMUX_PINCM49);
```

```
    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM26,  
IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM18,  
IOMUX_PINCM18_PF_GPIOB_DIO05, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM28,  
IOMUX_PINCM28_PF_GPIOB_DIO11, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
    DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINCM49,  
IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,  
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,  
DL_GPIO_WAKEUP_DISABLE);
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_9);
```

```
    DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_5);
```

```

DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_11);
DL_GPIO_clearPins(GPIOB, DL_GPIO_PIN_21);


DL_Timer_enablePower(TIMG12);
DL_Timer_ClockConfig config;
config.clockSel = DL_TIMER_CLOCK_BUSCLK;
config.divideRatio = DL_TIMER_CLOCK_DIVIDE_1;
config.prescale = 0;
DL_Timer_setClockConfig(TIMG12, &config);
DL_Timer_TimerConfig timerConfig;
timerConfig.timerMode = DL_TIMER_TIMER_MODE_PERIODIC_UP;
timerConfig.period = -1;
timerConfig.startTimer = DL_TIMER_START;
timerConfig.genIntermInt = DL_TIMER_INTERM_INT_DISABLED;
timerConfig.counterVal = 0;
DL_Timer_initTimerMode(TIMG12, &timerConfig);


while (testPB9 < 5) {
    printf("Test %d starting now ", testPB9 + 1);
    startPB9 = DL_Timer_getTimerCount(TIMG12);
    while (countPB9 < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prevPB9 !=
0) {

            countPB9++;
            prevPB9 = 0;

```

```

        DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

    }

    if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_9) && prevPB9 !=
1) {

        countPB9++;

        prevPB9 = 1;

        DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M26, IOMUX_PINCM26_PF_GPIOB_DIO09, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

    }

}

endPB9 = DL_Timer_getTimerCount(TIMG12);

countPB9 = 0;

prevPB9 = -1;

testPB9++;

frequencySumPB9 += (endPB9 - startPB9) / 60;

printf("%dhz\n", (endPB9 - startPB9) / 60);

delay_cycles(32000000);

}

averageFrequencyPB9 = frequencySumPB9 / 5;

printf("Average Frequency PB9: %dhz\n: ", averageFrequencyPB9);

```

```

while (testPB5 < 5) {
    printf("Test %d starting now ", testPB5 + 1);
    startPB5 = DL_Timer_getTimerCount(TIMG12);
    while (countPB5 < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_5) && prevPB5 !=
0) {

            countPB5++;

            prevPB5 = 0;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M18, IOMUX_PINCM18_PF_GPIOB_DIO05, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

        }

        if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_5) && prevPB5!= 1)
{

            countPB5++;

            prevPB5 = 1;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M18, IOMUX_PINCM18_PF_GPIOB_DIO05, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

        }

    }

    endPB5 = DL_Timer_getTimerCount(TIMG12);
    countPB5 = 0;
    prevPB5 = -1;
    testPB5++;

    frequencySumPB5 += (endPB5 - startPB5) / 60;
}

```

```

    printf("%dhz\n", (endPB5 - startPB5) / 60);

    delay_cycles(32000000);
}

averageFrequencyPB5 = frequencySumPB5 / 5;
printf("Average Frequency PB5: %dhz\n: ", averageFrequencyPB5);

while (testPB11 < 5) {
    printf("Test %d starting now ", testPB11 + 1);
    startPB11 = DL_Timer_getTimerCount(TIMG12);
    while (countPB11 < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_11) &&
prevPB11 != 0) {
            countPB11++;
            prevPB11 = 0;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M28, IOMUX_PINCM28_PF_GPIOB_DIO11, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);
        }
        if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_11) && prevPB11!=
1) {
            countPB11++;
            prevPB11 = 1;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M28, IOMUX_PINCM28_PF_GPIOB_DIO11, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

```

```

    }
}

endPB11 = DL_Timer_getTimerCount(TIMG12);
countPB11 = 0;
prevPB11 = -1;
testPB11++;

frequencySumPB11 += (endPB11 - startPB11) / 60;
printf("%dhz\n", (endPB11 - startPB11) / 60);

delay_cycles(32000000);
}

averageFrequencyPB11 = frequencySumPB11 / 5;
printf("Average Frequency PB11: %dhz\n: ", averageFrequencyPB11);

while (testPB21 < 5) {
    printf("Test %d starting now ", testPB21 + 1);
    startPB21 = DL_Timer_getTimerCount(TIMG12);
    while (countPB21 < 20) {
        if (!DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_21) &&
prevPB21 != 0) {
            countPB21++;
            prevPB21 = 0;

            DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M49, IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_UP, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

```

```

    }
    if (DL_GPIO_readPins(GPIOB, DL_GPIO_PIN_21) && prevPB21!=
1) {

        countPB21++;

        prevPB21 = 1;

        DL_GPIO_initPeripheralInputFunctionFeatures(IOMUX_PINC
M49, IOMUX_PINCM49_PF_GPIOB_DIO21, DL_GPIO_INVERSION_DISABLE,
DL_GPIO_RESISTOR_PULL_DOWN, DL_GPIO_HYSTERESIS_DISABLE,
DL_GPIO_WAKEUP_DISABLE);

    }

}

endPB21 = DL_Timer_getTimerCount(TIMG12);
countPB21 = 0;
prevPB21 = -1;
testPB21++;

frequencySumPB21 += (endPB21 - startPB21) / 60;
printf("%dhz\n", (endPB21 - startPB21) / 60);

delay_cycles(32000000);

}

averageFrequencyPB21 = frequencySumPB21 / 5;
printf("Average Frequency PB21: %dhz\n: ", averageFrequencyPB21);

while (1) {

```

}

}